

Задача о минимальном доминирующем множестве в графах:

NP-трудность и точный алгоритм

Карлов Николай

Группа: B05-328

Аннотация

Аннотация. Данная работа посвящена изучению задачи поиска наименьшего доминирующего множества в произвольном графе. Доминирующее множество — это такое подмножество вершин графа, что каждая вершина графа либо входит в это множество, либо смежна с какой-либо вершиной из него. Задача нахождения доминирующего множества минимального размера является одной из классических NP-трудных задач теории графов (это также доказано в работе).

В рамках работы представлено два основных результата. Во-первых, строго доказана NP-трудность данной задачи путём сведения к ней известной NP-полной задачи. Во-вторых, реализован точный алгоритм поиска наименьшего доминирующего множества, основанный на методах оптимизации переборных, асимптотическая сложность которого не превышает $O(c^n)$ для $c < 1.9$. Эффективность предложенного алгоритма подтверждается его сравнительным анализом с наивным переборным подходом.

1 Введение

1.1 Постановка задачи

Задача о минимальном доминирующем множестве (Minimum Dominating Set, MDS) формулируется следующим образом: для заданного графа $G = (V, E)$ требуется найти подмножество вершин $D \subseteq V$ минимального размера, такое что каждая вершина $v \in V$ либо принадлежит D , либо смежна с хотя бы одной вершиной из D .

1.2 Исторический контекст и значимость

Задача MDS является одной из классических NP-трудных задач теории графов. В списке 21 NP-полной задачи Ричарда Карпа 1972 года [3] содержится задача SET COVER; именно за счёт сведения к ней мы будем доказывать NP-трудность MDS. Задача имеет многочисленные приложения в теории сетей, проектировании микросхем, распределённых системах, биоинформатике и других разделах, где возникают графовые структуры.

1.3 Обзор существующих результатов

Тривиальный алгоритм полного перебора всех подмножеств вершин имеет сложность $O(2^n \cdot \text{poly}(n))$. В работе Грандони [1] был предложен алгоритм со сложностью $O(1.8021^n)$ (в его первой работе на самом деле был описан алгоритм решения Set Cover за $O(1.3424^n)$),

что стало существенным улучшением. Алгоритм основан на сведении к задаче о минимальном покрытии множеств и использовании техники ветвей и границ с нетривиальной мерой сложности, именно на основе этой статьи мы и будем доказывать корректность и асимптотику алгоритма.

1.4 Цели и структура работы

Основные цели данной работы:

1. Представить полное доказательство NP-трудности задачи MDS.
2. Описать и проанализировать(с подробными доказательствами) алгоритм Грандони.
3. Реализовать алгоритм и проверить его эффективность на практике.
4. Сравнить теоретические оценки с экспериментальными результатами.

Работа структурирована следующим образом: в разделе 2 вводятся основные определения, раздел 3 содержит доказательство NP-трудности, раздел 4 описывает алгоритм и его анализ, раздел 5 посвящён реализации и экспериментам.

2 Основные определения и обозначения

2.1 Определения из теории графов

Определение 1 (Соседи и замкнутая окрестность). Для вершины $v \in V$:

- Множество соседей: $N(v) = \{u \in V \mid \{u, v\} \in E\}$
- Замкнутая окрестность: $N[v] = \{v\} \cup N(v)$

Определение 2 (Доминирующее множество). Подмножество вершин $D \subseteq V$ называется доминирующим множеством графа G , если

$$\forall v \in V : (v \in D) \vee (\exists u \in D : \{u, v\} \in E).$$

Эквивалентно: $\bigcup_{v \in D} N[v] = V$.

Определение 3 (Задача MIN-DOMINATING-SET). Для заданного графа $G = (V, E)$ найти доминирующее множество минимальной мощности.

3 Доказательство NP-трудности задачи

В данном разделе мы будем анализировать не саму задачу(так как она оптимизационная), а её decision-версию. Тогда когда мы докажем, что decision-версия NP-трудна, мы получим, что она элементарно сводится к нашей задаче(просто найти ответ через нашу задачу и сравнить его с k из decision задачи), таким образом мы докажем NP-трудность MDS.

3.1 Принадлежность NP

Теорема 1. Задача DOMINATING SET принадлежит классу NP.

Доказательство. Покажем, что существует недетерминированная машина Тьюринга, которая разрешает задачу DOMINATING SET за полиномиальное время, или, что равносильно, существует детерминированный полиномиальный верификатор для сертификата.

Рассмотрим детерминированный алгоритм (верификатор) V , который получает на вход:

1. Исходный граф $G = (V, E)$ и число $k \in \mathbb{N}$ (экземпляр задачи);
2. Подмножество вершин $D \subseteq V$ в качестве сертификата.

Алгоритм V выполняет следующую проверку:

1. Проверяет, что $|D| \leq k$.
2. Для каждой вершины $v \in V$ проверяет выполнение условия доминирования:

$$(v \in D) \vee (\exists u \in D: \{u, v\} \in E).$$

Если оба условия выполнены, алгоритм принимает вход; иначе отвергает.

Корректность: По определению, алгоритм V принимает вход тогда и только тогда, когда предъявленное множество D является доминирующим множеством графа G размера не более k , а значит работает корректно.

Полиномиальность времени работы: Проверка первого условия выполняется за $O(|V|)$. Для проверки второго условия для каждой вершины $v \in V$ достаточно проверить принадлежность v множеству D (за $O(1)$) или существование соседа из D . Проверку существования соседа можно выполнить за $O(\deg(v))$, просмотрев список смежности вершины v . Суммируя по всем вершинам, получаем время работы $O(|V| + |E|)$, что полиномиально относительно размера входа.

Таким образом, задача DOMINATING SET принадлежит классу NP. $\square$

3.2 Сведение от задачи VERTEX-COVER

Теорема 2. Задача DOMINATING-SET является NP-полной.

Доказательство. **Часть 1: Построение сведения.**

Пусть дан экземпляр задачи VERTEX-COVER: граф $G = (V, E)$ и число k . Построим граф $G' = (V', E')$ следующим образом:

1. Определим множество изолированных вершин: Пусть $I = \{v \in V \mid \deg(v) = 0\}$.
2. Вершины: $V' = (V \setminus I) \cup E$ (все вершины (кроме изолированных) и все рёбра G становятся вершинами G')
3. Рёбра: $E' = E \cup \{(e, u), (e, v) \mid e = \{u, v\} \in E\}$

Часть 2: Доказательство эквивалентности.

Утверждаем, что в G существует вершинное покрытие размера не более k тогда и только тогда, когда в G' существует доминирующее множество размера не более k .

($\Rightarrow$) Пусть $C \subseteq V$ — вершинное покрытие G размера $|C| \leq k$. Покажем, что C задаёт доминирующее множество в G' :

1. Для любой исходной вершины $v \in V$:
 - Если $v \in C$, то она доминируется сама собой.
 - Если $v \notin C$, то поскольку C — вершинное покрытие, существует ребро $\{u, v\} \in E$ с $u \in C$, значит v доминируется u .
2. Для каждой вершины-ребра $e = \{u, v\} \in E$:
 - Поскольку C покрывает все рёбра, хотя бы один из концов u или v принадлежит C .
 - Значит, e смежна с вершиной из C (по построению G').

($\Leftarrow$) Пусть $D \subseteq V'$ — доминирующее множество в G' размера $|D| \leq k$. Построим вершинное покрытие $C \subseteq V$:

1. Инициализируем $C = \emptyset$.
2. Для каждой вершины $x \in D$:
 - Если $x \in V$, добавляем x в C .
 - Если $x \in E$ (т.е. $x = \{u, v\}$), выбираем произвольный конец, например u , и добавляем u в C .

Покажем, что C является вершинным покрытием:

- По построению $|C| \leq |D| \leq k$.
- Рассмотрим произвольное ребро $e = \{u, v\} \in E$:
 - * Вершина e в G' должна быть доминирована множеством D .
 - * Если $e \in D$, то в C лежит один из его концов, следовательно это ребро покрыто
 - * Иначе, если $e \notin D$, в D лежит элемент с которым у нашего e есть ребро в G' , по построению, это либо u либо v , давайте для простоты считать, что это u , тогда по построению $u \in C$, следовательно это ребро покрыто

Преобразование $(G, k) \rightarrow G'$ выполняется за полиномиальное время: $|V'| = |V| + |E|$, $|E'| = 3|E|$.

Таким образом, VERTEX-COVER $\leq_p$ DOMINATING-SET, что доказывает NP-полноту последней.

□

Следствие 1. Задача MINIMUM-DOMINATING-SET является NP-трудной.

Доказательство. как мы уже говорили выше, DOMINATING-SET очевидно сводится к MINIMUM-DOMINATING-SET, что и доказывает NP-трудность нашей задачи

□

4 Алгоритм поиска минимального доминирующего множества

Данный раздел основан на работе Грандони [1].

4.1 Сведение к задаче Set Cover

Лемма 1 (Сведение к Set Cover). *Задача о минимальном доминирующем множестве в графе $G = (V, E)$ с $n = |V|$ сводится к задаче о минимальном покрытии множеств с размерностью $k = 2n$.*

Доказательство. Построим экземпляр задачи Set Cover:

- Универсум $\mathcal{U} = V$ (вершины графа).
- Семейство множеств $\mathcal{S} = \{S_v \mid v \in V\}$, где $S_v = N[v] = \{v\} \cup N(v)$.

Тогда $D \subseteq V$ является доминирующим множеством в G тогда и только тогда, когда $\{S_v \mid v \in D\}$ является покрытием $\mathcal{U}$. $\square$

4.2 Основные правила редукции

Для формального описания алгоритма определим операцию удаления элементов. Для коллекции множеств $\mathcal{S}$ и множества элементов $R \subseteq \mathcal{U}$ положим:

$$\text{del}(\mathcal{S}, R) = \{S' \neq \emptyset \mid S' = S \setminus R, S \in \mathcal{S}\}.$$

То есть из каждого множества в $\mathcal{S}$ удаляются все элементы из R , а получившиеся пустые множества отбрасываются.

Лемма 2 (Правила редукции для Set Cover). *Для экземпляра Set Cover $(\mathcal{U}, \mathcal{S})$:*

1. **(Доминирование множеств)** *Если существует $S, R \in \mathcal{S}$ такой что $S \subset R$, то существует оптимальное покрытие без S . В частности:*

$$\text{msc}(\mathcal{S}) = \text{msc}(\mathcal{S} \setminus \{S\}).$$

2. **(Обязательный элемент)** *Если существует элемент $u \in \mathcal{U}$, принадлежащий только одному множеству $S \in \mathcal{S}$, то S обязательно входит в оптимальное покрытие. В частности:*

$$\text{msc}(\mathcal{S}) = 1 + \text{msc}(\text{del}(\mathcal{S} \setminus \{S\}, S)).$$

3. **(Ветвление)** *Для любого множества $S \in \mathcal{S}$:*

$$\text{msc}(\mathcal{S}) = \min \{ \text{msc}(\mathcal{S} \setminus \{S\}), 1 + \text{msc}(\text{del}(\mathcal{S} \setminus \{S\}, S)) \}.$$

Доказательство. Краткое обоснование:

1. Если $S \subset R$, то любое покрытие, содержащее S , останется покрытием, если заменить S на R . Следовательно, S можно исключить из рассмотрения без потери оптимальности.
2. Элемент u может быть покрыт только множеством S . Поэтому S обязано входить в любое покрытие. После его включения все элементы из S считаются покрытыми и могут быть удалены.
3. Рассмотрим два непересекающихся случая: оптимальное покрытие не содержит S (первый вариант) или содержит S (второй вариант, где после включения S удаляем покрытые им элементы).

$\square$

Алгоритм представляет собой прямую рекурсивную реализацию этих правил. Если применимо правило (1) или (2), задача упрощается без ветвления. Иначе выбирается произвольное (в оригинальной работе — максимальное по размеру) множество S , и алгоритм ветвится, рассматривая два случая из правила (3).

4.3 Анализ сложности: теорема о времени работы

Ключевой идеей анализа является введение *меры сложности* (measure) задачи — её *размерности*.

Определение 4 (Размерность экземпляра). Для экземпляра *Set Cover* $(\mathcal{U}, \mathcal{S})$ его размерностью называется величина $k = |\mathcal{S}| + |\mathcal{U}|$.

Напомним, что для экземпляра, полученного из графа с n вершинами по Лемме 1, размерность равна $k = 2n$.

Теорема 3 (Время работы алгоритма для Set Cover). Алгоритм, основанный на рекурсивном применении правил Леммы 2, решает задачу SET COVER за время $O(1.3803^k)$, где k — размерность экземпляра.

Доказательство. Обозначим через $T(k)$ максимальное время работы алгоритма на экземпляре размерности k . Время, затрачиваемое на обработку экземпляра до рекурсивных вызовов (проверка правил, выбор множества), полиномиально относительно k ; обозначим его $p(k)$. Тогда $T(k)$ можно представить как произведение числа подзадач на полиномиальное время их обработки. Введём для анализа вспомогательную функцию.

Определение. Пусть $N_h(k)$ обозначает максимальное количество подзадач размерности в точности h , которые могут быть порождены алгоритмом при решении исходной задачи размерности k (рассматриваются только листья дерева рекурсии — подзадачи, в которых тривиально определяется ответ).

Тогда общее число всех подзадач (листьев) $N(k)$ равно $\sum_{h=0}^k N_h(k)$, а время работы оценивается как $T(k) \leq N(k) \cdot p(k)$.

Шаг 1: Рекуррентное соотношение для $N_h(k)$. Рассмотрим, как порождаются подзадачи размерности h из задачи размерности k .

- Если применимо правило (1) или (2) (редукция без ветвления), то задача размерности k порождает единственную подзадачу размерности $k' \leq k - 1$. Следовательно, все подзадачи размерности h , порождённые из исходной, должны быть порождены из этой одной подзадачи:

$$N_h(k) \leq N_h(k - 1).$$

- Иначе применяется правило (3) (ветвление) по некоторому множеству S , $|S| \geq 2$. Алгоритм порождает две подзадачи:

1. $\mathcal{S}_1 = \mathcal{S} \setminus \{S\}$ размерности $k_1 = k - 1$ (удалено одно множество).
2. $\mathcal{S}_2 = \text{del}(\mathcal{S} \setminus \{S\}, S)$.

Размерность k_2 второй подзадачи удовлетворяет неравенству $k_2 \leq k - 4$, так как мы удаляем множество S наибольшего размера, и если его размер $|S| \geq 3$, то тогда очевидно, а если $|S| = 2$, то у нас есть какое-то множество размера ≤ 2 пересекающееся с S (так как S наибольшее и мы не удалили S по первому правилу), таким образом после удаления S , это множество станет размера ≤ 1 и сразу будет удалено по одному из правил редукции, таким образом получили ту же оценку. Поэтому любая подзадача размерности h , порождённая из исходной, должна быть порождена либо из первой, либо из второй ветви:

$$N_h(k) \leq N_h(k - 1) + N_h(k - 4).$$

Таким образом, для всех $k > h$ выполняется рекуррентное неравенство:

$$N_h(k) \leq N_h(k-1) + N_h(k-4). \quad (1)$$

Для граничных условий очевидно: $N_h(k) = 0$ при $h > k$, и $N_h(h) = 1$ (сама задача размерности h является подзадачей).

Шаг 2: Доказательство оценки $N_h(k) \leq c^{k-h}$. Докажем по индукции по k , что для всех $h \leq k$ выполняется $N_h(k) \leq c^{k-h}$, где $c \approx 1.3802... < 1.3803$ — наибольший вещественный корень уравнения $x^4 - x^3 - 1 = 0$.

- **База:** При $k = h$ имеем $N_h(h) = 1 = c^0 = c^{h-h}$.
- **Индукционный переход:** Предположим, что для всех $k' < k$ оценка верна. Рассмотрим $k > h$.

- Если из задачи размерности k не происходит ветвления, то $N_h(k) \leq N_h(k-1)$. По предположению индукции $N_h(k-1) \leq c^{(k-1)-h} = c^{k-h-1} < c^{k-h}$.
- Если происходит ветвление, то по (1) и предположению индукции:

$$N_h(k) \leq N_h(k-1) + N_h(k-4) \leq c^{k-1-h} + c^{k-4-h} = c^{k-h} (c^{-1} + c^{-4}).$$

Заметим, что c удовлетворяет уравнению $c^4 = c^3 + 1$. Поделив на c^4 , получим $1 = c^{-1} + c^{-4}$. Следовательно, выражение в скобках равно 1, и мы получаем:

$$N_h(k) \leq c^{k-h}.$$

Оценка $N_h(k) \leq c^{k-h}$ доказана.

Шаг 3: Оценка общего числа подзадач $N(k)$ и времени работы $T(k)$. Общее число подзадач (листьев дерева рекурсии) оценивается суммированием по всем h :

$$N(k) = \sum_{h=0}^k N_h(k) \leq \sum_{h=0}^k c^{k-h} = \sum_{i=0}^k c^i \quad (\text{замена } i = k - h).$$

Последняя сумма является суммой геометрической прогрессии:

$$\sum_{i=0}^k c^i = \frac{c^{k+1} - 1}{c - 1} = O(c^k).$$

Поскольку $c < 1.3803$, имеем $N(k) = O(1.3803^k)$.

Каждая подзадача обрабатывается за полиномиальное время $p(k)$, не считая времени на рекурсивные вызовы. Следовательно, общее время работы алгоритма:

$$T(k) \leq N(k) \cdot p(k) = O(1.3803^k) \cdot p(k) = O(1.3803^k).$$

□

Следствие 2 (Время работы для доминирующего множества). *Применяя алгоритм к экземпляру Set Cover, полученному из графа с n вершинами по Лемме 1 (где $k = 2n$), получаем алгоритм, решающий задачу о минимальном доминирующем множестве за время $O(1.3803^{2n}) = O(1.9053^n)$.*

Доказательство. Подставляя $k = 2n$ в оценку Теоремы 3, получаем $O(1.3803^{2n})$. Вычислим основание степени:

$$1.3803^2 = (1.3803)^2 \approx 1.9053.$$

Таким образом, итоговая сложность равна $O(1.9053^n)$.

□

Данный алгоритм использует полиномиальную память. В следующем подразделе рассматривается его улучшенная версия с мемоизацией.

4.4 Алгоритм с мемоизацией и улучшенная асимптотика

Модифицируем базовый алгоритм, добавив хранение решений уже решённых подзадач в некоторую базу данных (`unordered_map`, `map` или другой) (мемоизация). Это гарантирует, что каждая уникальная подзадача будет решена не более одного раза. Но появляется сильный минус - экспоненциальная память.

Теорема 4 (Время работы алгоритма с мемоизацией). *Алгоритм MSC, модифицированный путём добавления мемоизации (хранения решений подзадач), решает задачу SET COVER за время $O(1.3424^k)$, где k — размерность экземпляра. Эта оценка достигается за счёт использования экспоненциального объёма памяти.*

Доказательство. Пусть, как и прежде, $N_h(k)$ обозначает количество различных подзадач размерности h , которые могут встретиться при решении исходной задачи размерности k . Благодаря мемоизации, каждая из них решается ровно один раз.

Шаг 1: Две верхние оценки для $N_h(k)$.

1. **Комбинаторная оценка.** Подзадача однозначно определяется тем, какие из исходных k «объектов» (элементов множеств $\mathcal{S}$ и элементов универсума $\mathcal{U}$) были удалены в процессе её получения. Чтобы получить подзадачу размерности h , нужно удалить $k - h$ объектов. Следовательно,

$$N_h(k) \leq \binom{k}{h}.$$

2. **Рекурсивная оценка.** Из анализа базового алгоритма (Теорема 3) известно, что в дереве рекурсии (без мемоизации) число подзадач размерности h не превосходит c^{k-h} , где $c \approx 1.3803$ — корень уравнения $x^4 - x^3 - 1 = 0$. Поскольку мемоизация только уменьшает количество решаемых подзадач, эта оценка остаётся верной:

$$N_h(k) \leq c^{k-h}.$$

Таким образом, для каждого h справедливо:

$$N_h(k) \leq \min \left\{ \binom{k}{h}, c^{k-h} \right\}.$$

Шаг 2: Поиск оптимального разбиения суммы. Общее число подзадач $N(k) = \sum_{h=0}^k N_h(k)$ оценим, разделив сумму на две части при некотором t ($0 \leq t \leq k$):

$$N(k) \leq \sum_{h=0}^t \binom{k}{h} + \sum_{h=t+1}^k c^{k-h}.$$

Наша цель — выбрать t , минимизирующий эту верхнюю оценку. Заметим, что первое слагаемое растёт с увеличением t , а второе — убывает. Оптимальное t достигается, когда слагаемые сравнимы по порядку величины.

Шаг 3: Анализ при $t = \alpha k$. Будем искать t в виде $t = \alpha k$, где $0 < \alpha < 1$. Оценим оба слагаемых асимптотически при $k \rightarrow \infty$.

Для первого слагаемого используем оценку для суммы первых αk биномиальных коэффициентов. Для $\alpha < \frac{1}{2}$ наибольшее слагаемое — это $\binom{k}{\alpha k}$. По формуле Стирлинга,

$$\binom{k}{\alpha k} \sim \frac{1}{\sqrt{2\pi\alpha(1-\alpha)k}} \cdot \frac{k^k}{(\alpha k)^{\alpha k} ((1-\alpha)k)^{(1-\alpha)k}} = \frac{1}{\sqrt{2\pi\alpha(1-\alpha)k}} \cdot (\alpha^{-\alpha}(1-\alpha)^{-(1-\alpha)})^k.$$

Таким образом,

$$\sum_{h=0}^{\alpha k} \binom{k}{h} = O\left((\alpha^{-\alpha}(1-\alpha)^{-(1-\alpha)})^k\right).$$

Второе слагаемое — сумма геометрической прогрессии:

$$\sum_{h=\alpha k+1}^k c^{k-h} = \sum_{i=0}^{k-\alpha k-1} c^i = O(c^{k(1-\alpha)}).$$

Шаг 4: Определение α из условия баланса. Оптимальное α находится из условия, что оба слагаемых имеют одинаковый экспоненциальный порядок (с точностью до полиномиального множителя):

$$\alpha^{-\alpha}(1-\alpha)^{-(1-\alpha)} = c^{1-\alpha}. \quad (*)$$

Прологарифмируем (*) по основанию e :

$$-\alpha \ln \alpha - (1-\alpha) \ln(1-\alpha) = (1-\alpha) \ln c.$$

Переносим слагаемые, получаем уравнение для α :

$$\frac{\alpha}{1-\alpha} \ln \frac{1}{\alpha} + \ln(1-\alpha) = \ln c.$$

Подставляя $c = 1.3803$ и решая численно, находим $\alpha \approx 0.339$.

Шаг 5: Итоговая оценка. При найденном α оба слагаемых в оценке для $N(k)$ имеют один и тот же экспоненциальный порядок. Подставляя α в правую часть уравнения (*), вычисляем основание экспоненты:

$$c^{1-\alpha} \approx 1.3803^{1-0.339} \approx 1.3803^{0.661} \approx 1.3424.$$

Следовательно,

$$N(k) = O(1.3424^k).$$

Поскольку обработка каждой подзадачи (включая поиск в хеш-таблице) требует полиномиального времени $p(k)$, общее время работы равно $O(1.3424^k \cdot p(k)) = O(1.3424^k)$. $\square$

Следствие 3 (Время работы для доминирующего множества с мемоизацией). *Применяя модифицированный алгоритм с мемоизацией к экземпляру Set Cover, полученному из графа с n вершинами, получаем алгоритм, решающий задачу о минимальном доминирующем множестве за время $O(1.3424^{2n}) = O(1.8021^n)$.*

Доказательство. По Лемме 1 $k = 2n$. Подставляя в оценку Теоремы 4, получаем $O(1.3424^{2n})$. Вычисляем:

$$1.3424^2 = (1.3424)^2 \approx 1.8021.$$

Итоговая сложность равна $O(1.8021^n)$, что удовлетворяет условию задания ($c < 1.9$). $\square$

5 Реализация и экспериментальные результаты

5.1 Реализация алгоритма

Алгоритм был реализован на языке Python. Для представления множеств использовались битовые маски, что позволяет эффективно выполнять операции объединения, пересечения и проверки принадлежности. Также задача решалась не явно, в том смысле, что не было явного сведения к SET-COVER, а использовалась та же логика но сразу на задаче MDS.

```
1 class MinimumDominatingSetSolver:
2     def __init__(self, graph, adjacency_matrix=False, cache=True):
3         self.n = len(graph)
4
5         self.dominating_set = []
6
7         if adjacency_matrix:
8             for v in range(self.n):
9                 mask = 1 << v
10                for ind, neighbor in enumerate(graph[v]):
11                    if neighbor:
12                        mask |= 1 << ind
13                self.dominating_set.append(mask)
14        else:
15            for v in range(self.n):
16                mask = 1 << v
17                for neighbor in graph[v]:
18                    mask |= 1 << neighbor
19                self.dominating_set.append(mask)
20
21        self.use_cache = cache
22
23        if cache:
24            self._cache = {}
25            self._solve_recursive = self._solve_recursive_cached
26        else:
27            self._solve_recursive = self._solve_recursive_impl
28
29        def _solve_recursive_cached(self, U, available):
30            key = (U, available)
31            if key in self._cache:
32                return self._cache[key]
33
34            result = self._solve_recursive_impl(U, available)
35
36            self._cache[key] = result
37            return result
38
39        def solve(self) -> list:
40            if self.use_cache:
41                self._cache.clear()
42
43            U = (1 << self.n) - 1
44            available = (1 << self.n) - 1
```

```

45         return self._solve_recursive(U, available)
46
47     def _solve_recursive_impl(self, U, available):
48         if U == 0:
49             return []
50
51         #
52         new_available = self._removal_of_nested_sets(U, available)
53
54         if new_available != available:
55             return self._solve_recursive(U, new_available)
56
57         #
58         unique_covering_set = self.
59             _search_for_the_unique_covering_set(U, available)
60         if unique_covering_set != 0:
61             list_removed = []
62             while unique_covering_set:
63                 del_ind = (unique_covering_set & -unique_covering_set
64                     ).bit_length() - 1
65                 unique_covering_set &= unique_covering_set - 1
66
67                 U = self._del(U, del_ind)
68                 list_removed.append(del_ind)
69                 available &= ~(1 << del_ind)
70
71             return list_removed + self._solve_recursive(U, available)
72
73         #
74         ind_max = self._search_for_the_maximum_covering_set(U,
75             available)
76
77         first_branch = self._solve_recursive(U, available & ~(1 <<
78             ind_max))
79         second_branch = ([ind_max] +
80             self._solve_recursive(self._del(U, ind_max),
81                 available & ~(1 <<
82                     ind_max)))
83
84         if len(first_branch) > len(second_branch):
85             return first_branch
86         else:
87             return second_branch
88
89     def _removal_of_nested_sets(self, U, available):
90         temp_available = available
91         removed = 0
92
93         while temp_available:
94             v = (temp_available & -temp_available).bit_length() - 1
95             temp_available &= temp_available - 1
96
97             mask_v = self.dominating_set[v] & U

```

```

93         other_available = available & ~((1 << v) | removed)
94     while other_available:
95         u = (other_available & -other_available).bit_length()
96             - 1
97         other_available &= other_available - 1
98
99         mask_u = self.dominating_set[u] & U
100
101         if (mask_v & ~mask_u) == 0:
102             removed |= 1 << v
103             break
104
105     return available & ~removed
106
107 def _search_for_the_unique_covering_set(self, U, available):
108     temp_available = available
109     removed = 0
110
111     while temp_available:
112         v = (temp_available & -temp_available).bit_length() - 1
113         temp_available &= temp_available - 1
114
115         mask_v = self.dominating_set[v] & U
116
117         other_available = available & ~((1 << v) | removed)
118         while other_available:
119             u = (other_available & -other_available).bit_length()
120                 - 1
121             other_available &= other_available - 1
122
123             mask_u = self.dominating_set[u] & U
124
125             mask_v = (mask_v & ~mask_u)
126
127             if mask_v == 0:
128                 break
129
130             if mask_v != 0:
131                 removed |= 1 << v
132
133     return removed
134
135 def _del(self, U, ind_del):
136     return U & ~(self.dominating_set[ind_del])
137
138 def _search_for_the_maximum_covering_set(self, U, available):
139     temp_available = available
140
141     max_elem, ind_max = 0, 0
142
143     while temp_available:
144         v = (temp_available & -temp_available).bit_length() - 1

```

```

144         temp_available &= temp_available - 1
145
146     mask_v = self.dominating_set[v] & U
147     count_elem = mask_v.bit_count()
148
149     if count_elem > max_elem:
150         max_elem = count_elem
151         ind_max = v
152
153     return ind_max

```

Листинг 1: Реализация алгоритма на Python

5.2 Методика экспериментов

Для тестирования алгоритма использовались следующие типы графов:

1. Все маленькие не изоморфные графы(до 9 вершин)
2. Случайные графы Эрдёша—Реньи с различной плотностью.
3. Специальные графы: гиперкубы, решётки и короны
4. Реальные графы: в нашем случае это подграф римских дорог и дограф дружеских связей на Facebook

5.3 Результаты экспериментов

5.3.1 Первый эксперимент(все маленькие графы)

В данном эксперименте я запускал все три алгоритма(наивный, без кэша и с кэшем), на все возможных неизоморфных подграфах размера до 9(включительно). В результате я получил два следующих графика:

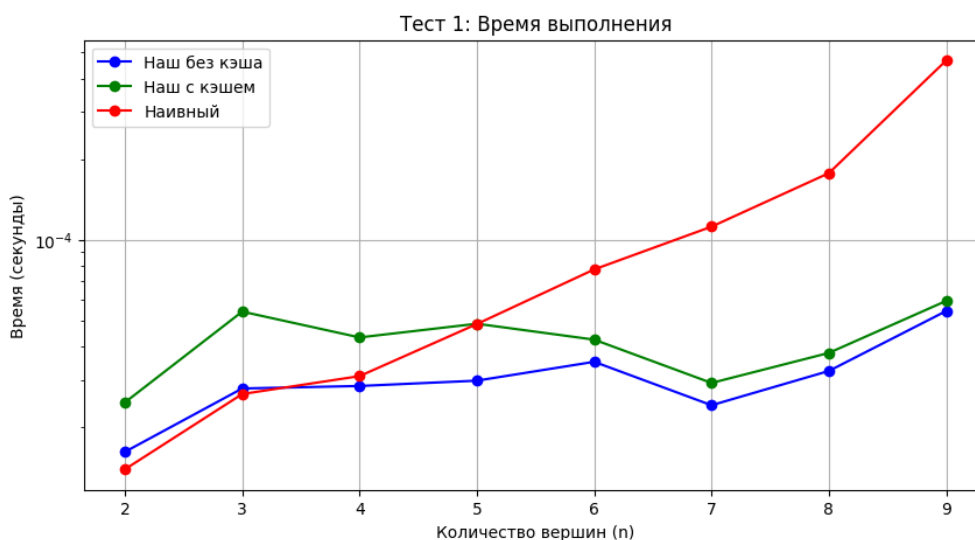


Рис. 1: Зависимость(log scale) времени выполнения от количества вершин на малых графах

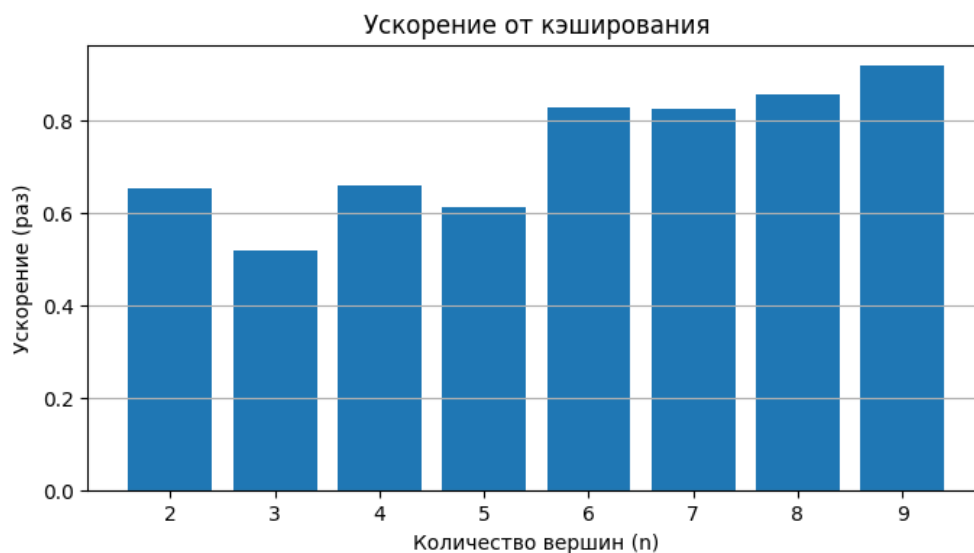


Рис. 2: Ускорение работы алгоритма за счёт кэширования на малых графах

5.3.2 Второй эксперимент($G(n, p)$ графы)

В данном эксперименте я запускал все три алгоритма(наивный, без кэша и с кэшем), на $G(n, p)$ графах с разными параметрами n и p . В результате я получил три следующих графика:

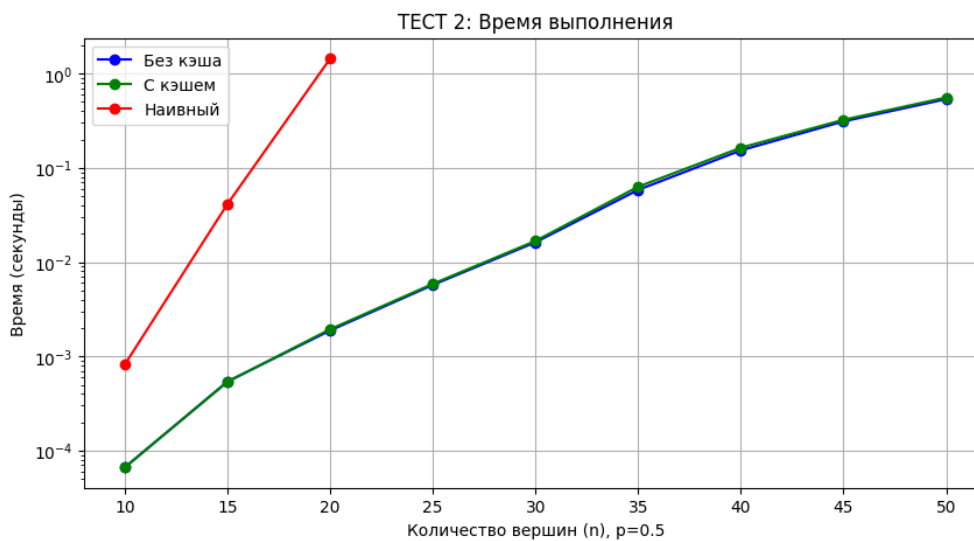


Рис. 3: Зависимость(log scale) времени выполнения от количества вершин на $G(n, p)$ графах

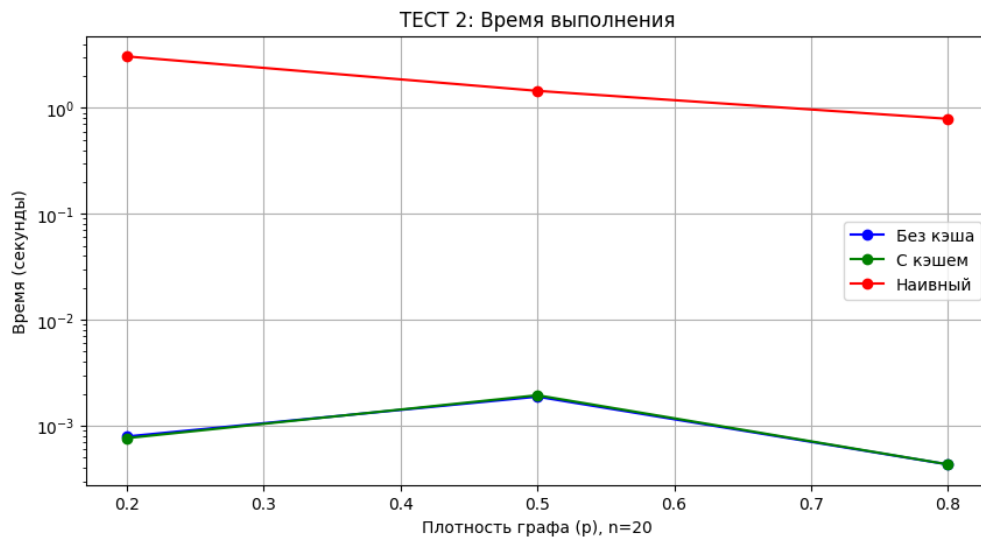


Рис. 4: Зависимость(log scale) времени выполнения от плотности графа на $G(n, p)$ графах(при фиксированном количестве вершин $n = 20$)

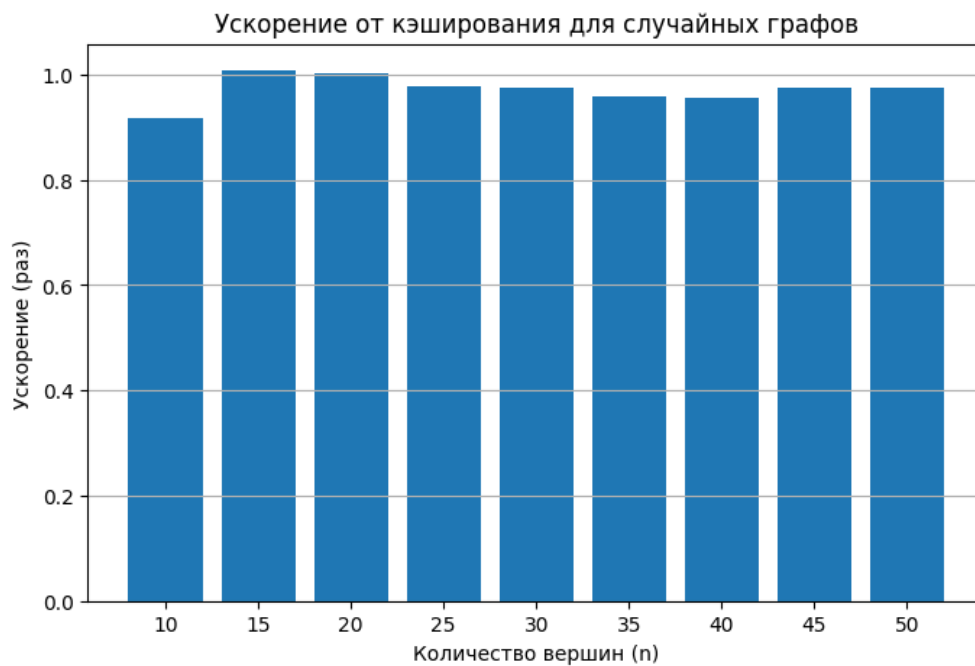


Рис. 5: Ускорение работы алгоритма за счёт кэширования на $G(n, p)$ графах

5.3.3 Третий эксперимент(Специальные графы)

В данном эксперименте я запускал два алгоритма(без кэша и с кэшем), на графах: гиперкубов, решёток и корон. Эти графы примечательны следующими особенностями:

1. Гиперкубы: проверяют работу на семетричных регулярных графах, где легко получить ответ, при этом вершины растут экспоненциально
2. Решётки: проверяют работу на планарных графах, с маленькими степенями вершин(из-за чего сложнее искать MDS), также этот тип графа схож с реальными сетевыми графами

3. Короны: проверяет работу на двудольных графах, также известен как пример сложного графа для поиска MDS

В результате я получил два следующих графика:

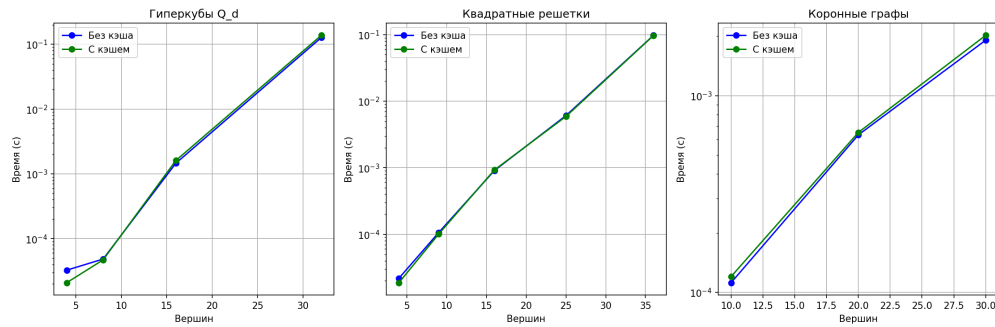


Рис. 6: Зависимость(log scale) времени выполнения от количества вершин на специальных графах

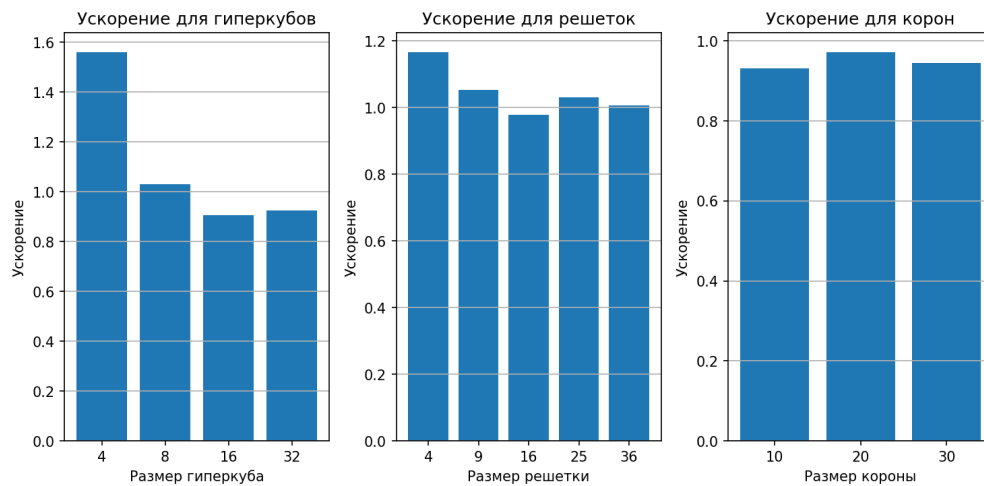


Рис. 7: Ускорение работы алгоритма за счёт кэширования на специальных графах

5.3.4 Четвёртый эксперимент(Реальные графы)

В данном эксперименте я запускал два алгоритма(без кэша и с кэшем), на графах: дорог Рима и связей друзей от Facebook. Эти графы примечательны следующими особенностями:

1. дороги Рима: в реальной жизни очень часто надо искать доминирующее множество на графах дорог, например для эффективного размещения экстренных спец служб
2. сеть друзей Facebook: для сети друзей очень часто нужно искать доминирующее множество, чтобы получить репрезентативную выборку людей для исследований

В результате я получил два следующих графика:

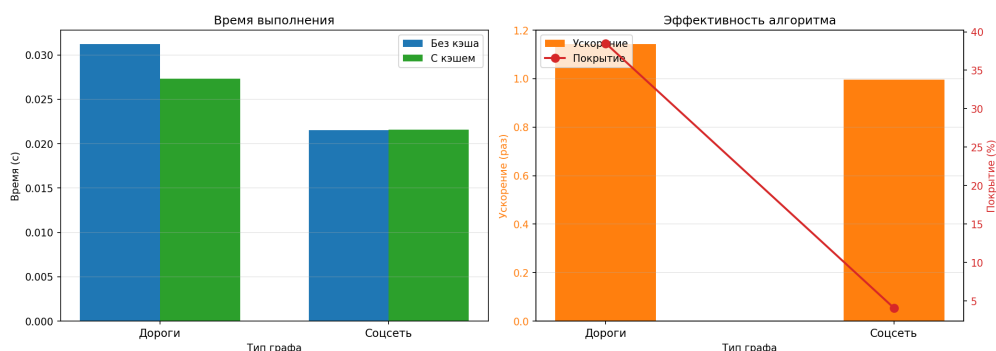


Рис. 8: Ускорение работы алгоритма за счёт кэширования на реальных графах

Параметр	Дороги	Соцсеть
Вершины (V)	200	1000
Рёбра (E)	508	9890
Ср. степень	5.1	19.8
D	77	41
Покрытие (%)	38.5	4.1
Время без кэша (с)	0.031	0.022
Время с кэшем (с)	0.027	0.022
Ускорение	1.1x	1.0x

Рис. 9: Таблица результатов тестирования на реальных данных

5.4 Анализ результатов

Результаты экспериментов следующие:

1. подтверждена экспоненциальная ассимптотика (в том смысле, что все три алгоритма экспоненциальные для некоторой константы c)
2. обе версии ускоренного MDS алгоритма работают лучше, чем наивная версия
3. версия без кэша часто показывает себя лучше, чем версия с кэшем, что не ожидаемо ввиду наших теоретических оценок, однако возможно это связано с маленькими графами в тестах (как мы видим на реальных графах с большим количеством вершин всё таки лучше версия с кэшем) и плохим кэшированием в питоне
4. ускоренные алгоритмы в некоторых случаях могут решать MDS на большом количестве вершин за адекватное время, что для наивного алгоритма было бы не возможно
5. ускоренные алгоритмы отлично показывают себя на реальных графах, на огромном для наивного алгоритма количестве вершин, что делает их очень важными для практики

6 Заключение

В данной работе были достигнуты следующие результаты:

1. Приведено полное доказательство NP-трудности задачи о минимальном доминирующем множестве путём сведения от задачи о вершинном покрытии.
2. Описан и проанализирован алгоритм Грандони, обеспечивающий асимптотическую сложность $O(1.8021^n)$.
3. Реализована версия алгоритма на языке Python и проведены эксперименты, подтверждающие его эффективность на практике.

Перспективы дальнейших исследований:

- Оптимизация реализации для работы с графами большего размера.
- Адаптация алгоритма для специальных классов графов (планарные, разреженные и т.д.).
- Разработка параллельной версии алгоритма.

Список литературы

- [1] Grandoni, F. (2005). A note on the complexity of minimum dominating set. *Journal of Discrete Algorithms*, 4(2), 209–214.
- [2] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [3] Karp, R. M. (1972). Reducibility among combinatorial problems. In *Complexity of Computer Computations* (pp. 85–103). Springer.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.